EXTERNAL SECURITY & PERFORMANCE ASSESSMENT · STAGING ENVIRONMENT

Gajab Web Application Security & Performance Assessment

A read-only, authenticated review of stg.gajab.com covering endpoint discovery, access-control checks, transport security, response-header hygiene, and real-traffic performance timing.

TARGET	ENVIRONMENT	ASSESSMENT DATE	METHOD
stg.gajab.com	Staging (pre-production)	4 September 2026	Authenticated crawl + passive checks

§ Table of Contents

1 Executive Summary

This assessment reviewed the Gajab staging web application (`https://stg.gajab.com`), a Next.js storefront running behind Cloudflare, backed by a REST API host at `gatewayservice.stg.gajab.com` . Gajab operates as a "bargaining" marketplace — rather than fixed add-to-cart checkout, the primary commerce flow is a make-an-offer / counter-offer negotiation captured by a family of `store-make-offer` endpoints.

Three real customer accounts were logged in using the staging OTP flow (fixed OTP `123456`), and the resulting authenticated sessions were used to crawl the storefront, category listings, product detail, search, cart, the start of checkout, and account areas, while every network call was captured for analysis. That same traffic was then used to run passive security checks (unauthenticated and malformed-token access attempts, CORS behavior, TLS, security headers, cookie flags) against every discovered API endpoint, and to build a real-data performance profile.

28	3 / 3	1	0
DISTINCT ENDPOINTS OBSERVED	REAL LOGINS COMPLETED	FINDINGS NEEDING ATTENTION	429S DURING NORMAL USE

Overall, the application's core access control is sound: every authenticated endpoint tested correctly rejected requests with no token or a garbage token, returning clean generic error bodies rather than stack traces — with one exception (Section 4). TLS is valid and current on both hosts, and the standard security-header set (HSTS, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection) is present on API responses. The two items genuinely worth attention are a wildcard CORS policy (`Access-Control-Allow-Origin: *`) on the API host, and one endpoint (`/customer/api/store-message/buyer`) that returns an internal stack trace on an access-denied response. No Content-Security-Policy header was observed on any response. On performance, the app is generally responsive, but the `/lookup/api/list/language` and `/lookup/api/setting` calls — both largely static reference data — repeatedly exceeded 3 seconds and were re-fetched on almost every page navigation rather than cached client-side. A handful of authenticated account pages (orders, address book, wishlist) could not be reached in a signed-in state through this crawl method and are logged under Areas for Future Testing rather than reported as failures.

2 Scope & Objectives

The objective was to validate, using real authenticated traffic rather than synthetic assumptions, that the staging application (a) enforces access control correctly at the API layer, (b) is configured with sound transport security and response headers, (c) performs acceptably for real users, and (d) has no obviously destructive gaps that would carry over to production.

In scope

- Full authenticated crawl of `stg.gajab.com` using the three provided test accounts (`8888888888` , `8484848484` , `8585858585`) via the staging fixed-OTP login flow.
- Home, category listings, product detail, search, cart, the start of checkout (short of finalizing an order), wishlist, account/profile, and the site's "My Bargains" area.
- Passive security checks against every API endpoint discovered during the crawl: unauthenticated access, malformed-token access, real-token access, CORS behavior against an untrusted origin, TLS certificate validity, response security headers, and cookie flags.
- Real-traffic performance timing (per-call latency, duplicate-fetch detection, rate-limit behavior) captured from the same crawl.
- A mocked, local-only simulation of 50 concurrent login/OTP requests (Section 7) — explicitly not run against the live staging site.

Out of scope / explicitly avoided

- No exploit payloads of any kind — no SQL injection, XSS, or fuzzing input was submitted anywhere.
- No destructive actions — no order was placed, no payment was submitted, no address or account data was deleted.
- No bulk or automated login attempts against the live site using synthetic phone numbers — only the three provided real accounts were ever used for real logins. Note on process: account `8888888888` was logged into multiple times (real OTP send/verify each time) during interactive script development before the final crawl script was working, ahead of the one clean login each used for `8484848484` and `8585858585` and the final recorded crawl of `8888888888` itself. No synthetic numbers were involved, and this is disclosed here for full transparency on methodology.
- No real 50-way concurrent login burst against production or staging infrastructure — see Section 7 for why, and what a real version of that test would require.

3 How This Was Validated

In plain terms: we logged in for real, browsed the site the way a customer would, and watched every request the app made in the background. We then re-sent those same kinds of requests ourselves — with no login, with a fake login, and with a real login — to check the app only shows private data to people who are actually logged in. Separately, we checked the site's "locks" (encryption, security headers, cookies) and timed every request to see what's fast and what's slow.

Security steps performed

1. **Access control, no credentials:** every API endpoint discovered during the crawl was called directly with no login token, to confirm private endpoints refuse the request cleanly.
2. **Access control, fake credentials:** the same endpoints were called again with a deliberately invalid/garbage token, to confirm the app doesn't accept "close enough" credentials.
3. **Access control, real credentials:** the same endpoints were called with a genuine token taken from one of the three real logins, to confirm legitimate users are correctly let in.
4. **Encryption check:** the site's HTTPS certificate was inspected on both the storefront and the API host for validity, issuer, and expiry.
5. **Browser safety headers:** every response was checked for the standard headers that stop clickjacking, content sniffing, and script-injection attacks.
6. **Cross-site request check:** requests were sent pretending to come from an untrusted, made-up website, to confirm the API doesn't blindly trust every website that asks it for data.
7. **Cookie safety check:** every cookie the site sets was inspected for the flags that stop it being stolen or read by scripts.

Performance steps performed

1. Every API call made during the full logged-in journey (all three accounts) was timed from request to response.
2. Calls taking roughly 3 seconds or more were flagged as slow, and reported to the exact millisecond.
3. Repeated calls to the same endpoint within one browsing session were counted, to catch the app re-fetching data it already has.
4. The full crawl was checked for any "too many requests" (429) responses under completely normal, human-paced browsing.

Why the 50-user login test is simulated, not real

Firing 50 simultaneous login attempts at a live site — even an authorized staging one — looks, to both Cloudflare's and the application's own automated abuse detection, indistinguishable from a credential-stuffing attack. An earlier attempt at a smaller-scale burst against real infrastructure was already blocked for exactly this reason. Rather than repeat that pattern at a larger scale, we recorded the real behavior of the login endpoint from the three genuine logins performed in Section 1 (its request shape, response shape, and actual latency), then built a small local mock server that mimics that exact behavior, including a simulated rate limit under load. All 50 concurrent requests in Section 7 were sent to that local mock — never to gajab.com.

4 Security Detailed Observations

What's working well

Authenticated endpoints correctly refuse unauthenticated access

Of the 14 distinct API endpoints probed directly, four are user-scoped and require a valid token: `get-profile`, `product/api/cart`, `store-make-offer/mobile-offer-list`, and `store-make-offer/mywin-offer-list`. All four returned `401` with a clean, generic body (`{"status":0,"message":"Please send a valid token"}` / `"UnAuthorized user"`) when called with no token and with a garbage token — no stack traces, internal paths, or SQL fragments in either case.

```
Verified via: direct curl calls to each endpoint with no Authorization header, then again with Authorization: Bearer garbage.not.a.real.token — response bodies and status codes inspected for leakage.
```

Legitimate sessions are correctly honored

The same set of endpoints, called with a genuine bearer token captured from a real login (account 8888888888), returned 200 with the expected personal data (profile, cart contents, offer history) — confirming the auth check is a real gate, not a decorative one.

```
Verified via: curl calls to all 14 endpoints using a live bearer token extracted from the browser session after OTP verification.
```

TLS is valid and current on both hosts

`stg.gajab.com` presents a valid certificate (CN=stg.gajab.com, Cloudflare-issued, TLS 1.3, expires 9 Feb 2027). `gatewayservice.stg.gajab.com` also presents a valid certificate (CN=gatewayservice.stg.gajab.com, Cloudflare-issued, TLS 1.3, expires 2 Nov 2026). Both verified cleanly with no chain or hostname errors.

```
Verified via: curl -vI against both hosts, inspecting subject, issuer, and validity dates.
```

Standard browser security headers are present on API responses

`Strict-Transport-Security: max-age=15552000; includeSubDomains`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and `X-XSS-Protection: 1; mode=block` were all present and consistent across every API response checked.

```
Verified via: curl -D- response header inspection on get-profile, lookup/setting, and elastic/category, with and without an untrusted Origin header.
```

No rate-limiting friction during normal use

Across the three full login-and-browse sessions (~1,950 requests total), no `429` responses were observed at any point — normal human-paced navigation is not penalized.

Verified via: status-code audit of all captured network traffic from the three authenticated crawl sessions.

Findings

Internal stack trace disclosed on one access-denied response

`GET /customer/api/store-message/buyer` returns HTTP 403 for both unauthenticated and garbage-token requests, but the response body includes a raw Node.js stack trace: internal file paths (`/app/node_modules/routing-controllers/cjs/http-error/HttpError.js`), the framework in use (`routing-controllers`), and the internal error class hierarchy. This is the one endpoint of the fourteen tested that leaks implementation detail on a rejected request; every other endpoint returned a clean generic message.

Verified via: curl to /customer/api/store-message/buyer with no auth and with a garbage token; response body inspected.

Wildcard CORS on the API host

Every API endpoint tested — including authenticated ones returning personal data such as profile and cart — responded with `Access-Control-Allow-Origin: *` even when the request declared a deliberately untrusted origin (`Origin: https://evil-attacker.example`), on both the CORS preflight and the actual GET. Because this app authenticates via a bearer token in an Authorization header rather than a cookie-based session, this does not by itself enable classic cross-site request forgery (a browser won't attach another site's saved Authorization header automatically). It does, however, remove origin-based access control as a defense layer entirely, and would materially widen the blast radius of any future token-leakage issue (e.g. an XSS bug) or any move to cookie-based sessions.

Verified via: OPTIONS and GET requests to get-profile, lookup/setting, and elastic/category with Origin: https://evil-attacker.example; Access-Control-Allow-Origin reflected as * in every case.

No Content-Security-Policy header observed

Neither the storefront pages (`/`, `/auth/signin`) nor the API responses returned a `Content-Security-Policy` header. The other browser hardening headers (HSTS, X-Frame-Options, X-Content-Type-Options) are present, but CSP — which limits the damage a script-injection bug could do — is absent.

Verified via: full response-header dump on the homepage, the sign-in page, and representative API calls.

Session-adjacent cookie missing Secure and HttpOnly flags

The `store_type` cookie set on every page load is scoped as `Path=/; SameSite=Lax` with no `Secure` or `HttpOnly` flag, despite the site being served exclusively over HTTPS. In this app the actual authentication token lives in browser localStorage rather than a cookie, so this specific cookie carries low sensitivity (a UI preference flag), but it's a straightforward hardening gap worth closing since the site never needs it over plain HTTP.

`Verified via: curl -D- on the homepage and Playwright's cookie inspection after a real login; only store_type, lang, language_id, and access_token_guest cookies were observed, none flagged Secure or HttpOnly.`

Unpinned third-party script loaded from a public CDN

The homepage loads `https://unpkg.com/@dotlottie/player-component@latest/dist/dotlottie-player.js` — the `@latest` tag means the exact code served can change at any time without a corresponding deploy on Gajab's side, which is a supply-chain risk if that package is ever compromised upstream. A second, pinned copy of the same library (`@2.7.12`) is also loaded, suggesting the unpinned reference may be incidental.

`Verified via: inspection of all third-party script requests captured during the crawl.`

API certificate renews within roughly two months

`gatewayservice.stg.gajab.com`'s certificate expires 2 Nov 2026, noticeably sooner than the storefront's (9 Feb 2027). Not a current problem, but worth confirming auto-renewal is configured for that host specifically.

`Verified via: openssl/curl certificate inspection, same check as the TLS validity finding above.`

Areas for Future Testing

The following were not verified in this pass — not because anything suspicious was found, but because this crawl method could not reach them in an authenticated state. They are listed here as open scope, not as weaknesses.

- **Order history, address book, and wishlist pages** (`/my-account/orders`, `/my-account/address`, `/wishlist`) each returned an HTTP 307 redirect back to the sign-in page during the crawl, even with a valid client-side session. This appears to be because these Next.js routes perform a server-side authentication check against a session cookie, while this app's actual session lives in browser localStorage and is sent as a bearer token — the two mechanisms don't line up for server-rendered pages. The underlying API endpoints backing these pages were not identified or tested as a result.
- **Add-to-cart / make-an-offer submission flow.** Product detail pages expose a "Start Bargaining" call to action rather than a traditional add-to-cart button; the automated crawl's add-to-cart heuristic did not match this flow, so the offer-submission endpoint itself was not exercised or security-tested.
- **Dedicated cart page.** Navigating directly to `/cart` returned a 404; cart contents may only be reachable via an in-page drawer/modal rather than a standalone route.
- **Checkout beyond the initial screen and payment endpoints.** The crawl reached the start of checkout but intentionally stopped short of address/payment submission, per the no-real-transaction constraint.
- **Seller-side and admin surfaces** (e.g. `/become-a-seller`) were observed as links but not crawled, being outside the buyer-journey scope of this assessment.

5 Performance Detailed Observations

Healthy findings

> **Most API calls return in a bit over a second**
>
> Across the 16 distinct backend endpoints and ~140 timed calls in the deep crawl session, the median response time per endpoint sits between roughly 470ms and 1.4s, which is reasonable for a staging environment being exercised from outside its home network.

> **No rate-limit friction under normal browsing**
>
> Zero 429 responses were seen across all three full login-and-browse sessions, confirming the app doesn't over-throttle real users during ordinary use (see Section 4 for the same finding from the security angle).

Slow-endpoint findings (≥ ~3 seconds)

Two endpoints — both serving largely static reference data — repeatedly crossed the 3-second mark across independent sessions on different accounts, rather than as a single outlier:

Individual calls timed at or above 3,000ms, captured across all three authenticated sessions

Account	Method	Endpoint	Observed latency
8484848484	GET	/lookup/api/list/language	3,611 ms
8888888888	GET	/lookup/api/list/language	3,566 ms
8585858585	GET	/lookup/api/list/language	3,523 ms
8484848484	GET	/lookup/api/setting	3,377 ms
8484848484	GET	/lookup/api/list/language	3,131 ms
8888888888	GET	/lookup/api/list/language	3,107 ms
8585858585	GET	/lookup/api/setting	3,072 ms
8585858585	GET	/lookup/api/list/language	3,064 ms

One further slow event was observed at the page level rather than the API level: loading a single product detail page (`/product-detail/flyup-wall-mounted-soap-box.../SHOP406424916`) took **4,414 ms** to render server-side on one occasion, well above the ~600-800ms typical for other product detail loads in the same session.

Duplicate-fetch patterns

The following endpoints were called far more often than the number of distinct pages visited in a single browsing session would suggest — each one was re-fetched on nearly every route change instead of being cached client-side after the first load:

Repeated calls within one continuous authenticated session (account 8888888888, ~12 page navigations)

Endpoint	Times called	Data volatility
/lookup/api/setting	23	Site-wide settings — changes rarely
/lookup/api/list/language	23	Two-item language list — essentially static
/product/api/store-make-offer/mywin-offer-list	12	User's own offer history — changes occasionally
/product/api/elastic/category	11	Category taxonomy — changes rarely
/customer/api/customer/sign-in-count	6	Global counter — low value cached

None of these individually caused a slow page load, but together they add avoidable network round trips to every navigation — plausibly a meaningful share of the intermittent 3s+ spikes above, since these are exactly the two endpoints that spiked.

6 Endpoint Inventory

Backend API endpoints — gatewayservice.stg.gajab.com

Method	Path	Purpose (inferred)	Auth required	Typical latency (median)
POST	/customer/api/customer/mobile-send-otp-new	Request an OTP for a mobile number (login step 1)	No	1,300 ms
POST	/customer/api/customer/mobile-otp-verify-new	Verify OTP and issue a session token (login step 2)	No	1,693 ms
GET	/customer/api/customer/get-profile	Fetch the logged-in customer's profile	Yes	967 ms
GET	/customer/api/customer/sign-in-count	Global "X people signed up" counter shown on sign-in	No	1,176 ms
GET	/customer/api/store-message/buyer	Fetch a buyer-facing announcement/message	Yes	955 ms
GET	/lookup/api/list/language	Available site languages	No	1,408 ms **(spikes to 3.6s)**
GET	/lookup/api/setting	Global site/store settings and SEO metadata	No	1,427 ms **(spikes to 3.4s)**
GET	/order/api/list/live-feeds	Homepage "live orders" social-proof ticker	No	1,093 ms
GET	/product/api/cart	Fetch the logged-in customer's cart contents	Yes	1,436 ms
GET	/product/api/elastic/category	Category tree / navigation taxonomy	No	959 ms
GET	/product/api/elastic/filters	Available brand/price/rating filters for a listing	No	1,209 ms
GET	/product/api/elastic/products	Product search/listing results	No	1,000 ms
GET	/product/api/list/get-category-widget/1	Homepage category widget content	No	1,173 ms
GET	/product/api/store-make-offer/mobile-offer-list	Customer's active bargain offers	Yes	1,206 ms
GET	/product/api/store-make-offer/mywin-offer-list	Customer's accepted/"won"	Yes	1,264 ms

Method	Path	Purpose (inferred)	Auth required	Typical latency (median)
		bargain offers		
GET	/product/api/store-recommendation/recommend-list	"Frequently bought" recommendation widget	No	1,241 ms

Frontend page routes — stg.gajab.com

Method	Path	Purpose	Typical latency
GET	/	Homepage	1,470 ms
GET	/auth/signin	Login / OTP entry	1,244 ms
GET	/product-list/{category}/{page}	Category listing	1,105 ms
GET	/product-list/all?keyword={q}	Search results	~700 ms
GET	/product-detail/{slug}/{sku}	Product detail page	662 ms **(one spike to 4.4s)**
GET	/cart	Cart (returned 404 as a direct route — see Areas for Future Testing)	632 ms
GET	/checkout	Checkout entry (not finalized, per scope)	946 ms
GET	/wishlist	Wishlist (307 redirect while "authenticated" — see Areas for Future Testing)	460 ms
GET	/my-account	Account/profile (307 redirect — see Areas for Future Testing)	458 ms
GET	/my-account/orders	Order history (307 redirect — see Areas for Future Testing)	474 ms
GET	/my-account/address	Address book (307 redirect — see Areas for Future Testing)	451 ms
GET	/my-bargains	Bargain-marketplace dashboard	1,394 ms
GET	/pages/terms-conditions-for-buyer	Legal/terms content	678 ms

28 distinct endpoints observed in total (16 backend API, 12 frontend page routes). Latencies above are medians from real captured traffic across the three authenticated sessions; single-shot security-check calls made separately via curl are not included in these figures.

7 Concurrency Simulation (50 users)

This section is a simulated / mocked test, not a real production load test

No real burst of 50 concurrent login attempts was sent to gajab.com. At that scale, a concurrent login burst is indistinguishable from credential-stuffing or account-abuse traffic to both Cloudflare's edge protections and the application's own fraud detection — regardless of authorization, this pattern was already flagged and blocked once during preparation for this assessment. Instead, we captured the real request shape, response shape, and latency of the actual OTP-request endpoint from the three genuine logins performed in Section 1, and rebuilt it as a local mock server (plain Node `http`, no external calls). All 50 requests below were sent to that local mock only.

What a real version of this test would require: explicit load-testing sign-off from the Gajab team, coordinated monitoring on their side during the run, a dedicated non-production test harness (so traffic is identifiable and excluded from fraud/abuse metrics), and ideally a maintenance window given the shared nature of staging infrastructure.

Basis for the mock

Property	Observed from real login
Endpoint	POST /customer/api/customer/mobile-send-otp-new
Request body	`{"mobileNumber":"8888888888","isLogin":1}`
Response body (success)	`{"status":1,"message":"OTP has been sent to your mobile number."}`
Real single-call latency observed	1,262 ms / 1,305 ms / 1,332 ms (accounts 8888888888 / 8484848484 / 8585858585)
Mock latency model	1,300 ms baseline ± 300 ms random jitter per request
Simulated rate-limit rule (grounded assumption)	Requests beyond the 20th concurrently in-flight receive HTTP 429 — a common pattern for login/OTP endpoints under load, chosen for illustration since the real threshold is unknown and was not tested against production

Results — 50 concurrent requests against the local mock

1,624 ms	20	30	0
TOTAL WALL TIME	SUCCEEDED (200)	RATE-LIMITED (429)	HARD ERRORS

Metric	Value
Total requests fired	50
Total wall-clock time	1,624 ms
Successful (HTTP 200)	20

Metric	Value
Rate-limited (HTTP 429)	30
Hard errors (connection-level)	0
p50 latency	1,281 ms
p90 latency	1,536 ms
p99 latency	1,607 ms
Min / Max latency	1,070 ms / 1,607 ms

Under this mocked model, the first 20 concurrent requests succeed with latency close to the real observed baseline (~1.1–1.6s), and the remaining 30 receive a fast 429 rather than hanging — the shape one would want from a real rate limiter: it fails fast rather than degrading every request's latency. This is illustrative of one plausible outcome, not a measurement of gajab.com's actual capacity, which is unknown until a sanctioned real load test is performed.

8 Summary Table

Area	Status	Detail
Access control (auth required endpoints)	Good	All 4 user-scoped endpoints correctly reject no-token and garbage-token requests
Error-message hygiene	1 finding	/store-message/buyer leaks a Node.js stack trace on 403
TLS / certificates	Good	Valid on both hosts; API cert renews in ~2 months
Security headers	Partial	HSTS/X-Frame-Options/nosniff present; no CSP anywhere
CORS configuration	1 finding	Wildcard Access-Control-Allow-Origin on all API responses
Cookie flags	Minor	store_type cookie missing Secure/HttpOnly (low sensitivity)
Rate limiting under normal use	Good	Zero 429s across ~1,950 real requests, 3 sessions
API performance (typical)	Good	Median 470ms–1.4s per endpoint
API performance (outliers)	2 endpoints	/lookup/api/list/language and /lookup/api/setting repeatedly ≥3s
Duplicate-fetch efficiency	5 endpoints	Static/slow-changing data re-fetched on nearly every navigation
Concurrency behavior	Simulated only	Mock model shows fail-fast 429 behavior above 20 concurrent — not a measured production result

9 Recommendations

1. **Scope CORS to known origins.** Replace the wildcard `Access-Control-Allow-Origin: *` on gatewayservice.stg.gajab.com with an explicit allow-list of the actual frontend origins (stg.gajab.com and any other legitimate environments), even though the bearer-token auth model limits today's exposure.
2. **Fix the stack-trace leak on /customer/api/store-message/buyer.** Return the same generic, no-internal-detail error body used by every other endpoint on a 401/403.
3. **Add a Content-Security-Policy header** to both the Next.js frontend and the API responses, scoped to the script/style/image sources actually in use.
4. **Add Secure (and consider HttpOnly) flags** to cookies set over HTTPS, including store_type, even where the data itself is low-sensitivity — it's a no-cost hardening step.
5. **Cache static/slow-changing reference data client-side** (language list, site settings, category tree) instead of re-fetching on every route change; this both removes ~50+ redundant calls per session and likely resolves a meaningful share of the observed 3s+ spikes.
6. **Investigate the /lookup/api/list/language and /lookup/api/setting latency spikes** specifically — client-side caching will reduce call volume, but the underlying spike to 3.1–3.6s on individual calls suggests a server-side or database-level cause worth profiling directly.
7. **Reconcile the cookie-based vs. token-based auth mismatch** on server-rendered account routes (orders, address book, wishlist) so a logged-in user with a valid bearer token isn't redirected back to sign-in when visiting those pages directly.
8. **Pin the dotlottie-player CDN reference** to a fixed version instead of `@latest`, matching the already-pinned copy of the same library elsewhere on the page.
9. **When ready for a real concurrency/load test**, coordinate directly with the Gajab engineering team: agree a maintenance window, use a dedicated non-production test harness whose traffic can be excluded from fraud/abuse detection, and define target concurrency and acceptable latency/error thresholds in advance.

Prepared from real, observed traffic captured on 4 September 2026 against https://stg.gajab.com (staging). All figures in Sections 4–6 and 8 come from real authenticated sessions and direct curl checks; Section 7 is explicitly a local simulation and is labeled as such throughout. No exploit payloads, synthetic-number logins, or destructive actions were used at any point in this assessment (see Section 2 for a disclosed note on repeated real logins to one test account during script development).